

LABORATORIO 16

Simulación de migración

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo. Las salidas son las de la solución ejecutada del repositorio.

Introducción

1 · El tema

En este laboratorio haces una migración completa, de principio a fin. Una tabla vive en una base de datos relacional, PostgreSQL, y hay que llevarla a una tabla Iceberg sin apagar el origen, porque el origen sigue recibiendo movimientos mientras dura la migración.

No aprendes una sentencia nueva. Juntas en un solo trabajo la lectura de otra base por JDBC, la carga inicial, la detección de cambios, la carga incremental con `MERGE` y la validación.

2 · El problema, en el almacén

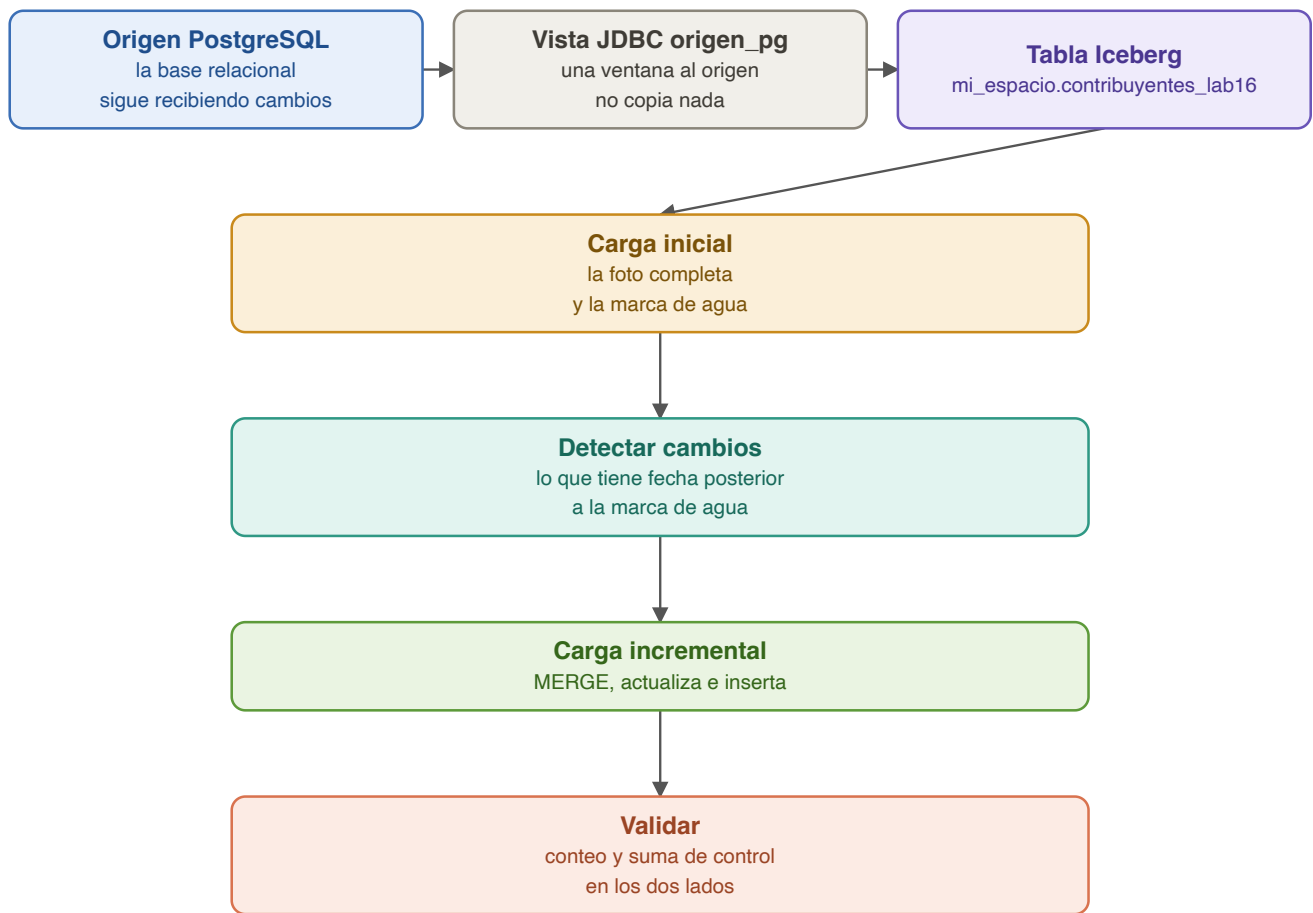
Un almacén se cambia de libreta. La libreta vieja está en el mostrador y el negocio sigue atendiendo, así que mientras alguien copia las páginas a la libreta nueva, en la vieja se siguen anotando ventas y correcciones. Si el que copia termina y se va, la libreta nueva nace atrasada. Y si cada noche vuelve a copiar la libreta entera desde la primera página, pasa la noche entera copiando cosas que ya tenía.

3 · El problema en términos técnicos

Una migración no es una copia única. Si copias la tabla entera y te vas, para cuando terminas el origen ya cambió. Y volver a copiar la tabla completa todas las noches no escala, porque en una institución son millones de filas y cientos de tablas.

Lo que se hace es copiar una vez todo y después solo lo que se movió. Para eso hace falta saber hasta dónde copiaste la primera vez, y hace falta una forma de meter lo nuevo sin duplicar lo que ya estaba. Al final hace falta una prueba de que los dos lados dicen lo mismo, y no la palabra de alguien que dice que salió bien.

4 · El diagrama



Origen PostgreSQL (azul). Es la base relacional donde vive hoy la tabla de contribuyentes. No se apaga durante la migración y sigue recibiendo altas y correcciones.

Vista JDBC origen_pg (gris). Es la forma en que Spark mira el origen. No copia nada, es una ventana que lee el origen cada vez que se consulta.

Tabla Iceberg (morado). Es el destino, la tabla nueva que queda en tu espacio de trabajo.

Carga inicial (amarillo). Copia todo una vez y anota la marca de agua, que es la fecha de actualización más nueva que alcanzó a traer.

Detectar cambios (verde agua). Pregunta al origen qué filas tienen una fecha de actualización posterior a la marca de agua. Eso es lo que falta.

Carga incremental (verde). Mete solo lo que falta con `MERGE`, que actualiza las filas que ya estaban e inserta las nuevas en una sola sentencia.

Validar (coral). Compara los dos lados con un conteo y con una suma de control. Si coinciden, la migración está validada.

5 · La solución

En el almacén, el que copia anota en un papel hasta qué hora alcanzó a copiar. La noche siguiente no vuelve a empezar desde la primera página. Busca en la libreta vieja solo lo que se anotó después de esa hora, lo pasa a la nueva, corrigiendo los renglones que ya tenía y agregando los que no, y al final cuenta los renglones de las dos libretas para comprobar que dicen lo mismo.

En la tecnología, ese papel es la **marca de agua**, la fecha de actualización más nueva de la carga inicial. La búsqueda es un `WHERE actualizado_en > marca`. El pase a la libreta nueva es un `MERGE INTO`, que decide fila por fila si actualiza o inserta. Y la comprobación es un conteo más una suma de control calculada igual en los dos lados.

6 · Los pasos

- **Paso 0.** La idea de la marca de agua, sin celdas.
- **Paso 1.** Abres el origen con una vista JDBC y miras sus veinte contribuyentes.
- **Paso 2.** Haces la carga inicial a una tabla Iceberg y anotas la marca de agua.
- **Paso 3.** Mueves el origen desde una terminal, como lo haría el sistema de la DGT.
- **Paso 4.** Detectas qué cambió desde la marca de agua.
- **Paso 5.** Cargas solo eso con `MERGE`.
- **Paso 6.** Validas con conteo y suma de control.
- **Paso 7.** Miras las páginas que dejó la migración.

Para que el laboratorio salga como en esta guía, el origen tiene que partir con veinte contribuyentes y sin cambios. Si ya lo corriste antes, déjalo como estaba con el comando de abajo, desde la carpeta del repositorio.

```
bash
bin/mover-origen-lab16.sh --reset
```

Paso 0 · Qué es migrar, y qué es la marca de agua

Este paso no tiene celdas. Es texto en el cuaderno, y aquí va explicado.

La pregunta. ¿Cómo se migra una tabla que no para de cambiar?

Por qué ahora. Todo el laboratorio se apoya en una sola idea, y conviene tenerla clara antes de tocar nada.

En el almacén. El que copia la libreta anota en un papel hasta qué hora alcanzó a copiar. Ese papel es lo que le permite, la noche siguiente, copiar solo lo nuevo.

La **marca de agua** es la fecha de actualización más nueva que alcanzaste a traer. Todo lo que en el origen tenga una fecha posterior es lo que falta. En este laboratorio la columna se llama `actualizado_en` y la mantiene el sistema de origen, que le pone la hora cada vez que una fila se crea o cambia. En una migración real, si esa columna no existe, lo primero que hay que conseguir del dueño del origen es que exista.

Paso 1 · Mirar el origen

Celda 1.1 · Entrar a tu espacio

La pregunta. ¿En qué espacio van a quedar las tablas que crees?

Por qué ahora. La tabla Iceberg de destino tiene que quedar en tu espacio de trabajo, `mi_espacio`. Esta celda te pone ahí.

En el almacén. Es pararte frente a tu propio estante antes de empezar a trabajar.

La sentencia, parte por parte.

- `%%sql` es la primera línea de toda celda SQL del cuaderno. Le dice al cuaderno que lo que sigue es SQL y no Python.
- `-- Celda 1.1` es un comentario. Todo lo que va después de dos guiones lo ignora el motor.
- `USE mi_espacio` deja a `mi_espacio` como el espacio por omisión. Desde aquí, una tabla nombrada sin espacio adelante se busca y se crea en `mi_espacio`.

```
sql
%%sql
-- Celda 1.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:11:35 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. La sentencia se ejecutó. Las tres primeras líneas son avisos de Spark al arrancar, no errores, y salen solo en la primera celda.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores, no todo lo que hace.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar eso. `sc` es el contexto de Spark y `setLogLevel` la función que cambia el nivel. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop escrita para este sistema operativo y que usa la versión en Java. Funciona igual. `26/09/25 18:11:35` es la fecha y la hora del aviso, año 2026, mes 09, día 25, en hora UTC.

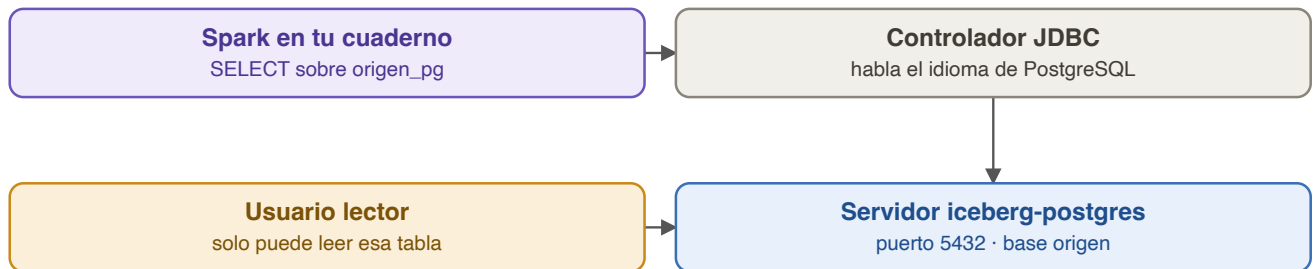
Listo. La sentencia se ejecutó. es lo que muestra el cuaderno cuando una sentencia no devuelve filas.

Celda 1.2 · Abrir el origen por JDBC

La pregunta. ¿Cómo lee Spark una tabla que vive en otra base de datos?

Por qué ahora. Antes de copiar nada hay que poder mirar el origen desde el mismo lugar donde vas a escribir el destino.

En el almacén. Es abrir una ventana hacia la libreta vieja, que sigue en el mostrador. Por la ventana se ve lo que dice, pero no se trae nada.



Spark en tu cuaderno (morado). Cada vez que una consulta nombra `origen_pg`, Spark va a buscar las filas al origen en ese momento.

Controlador JDBC (gris). JDBC es la forma estándar en que un programa Java conversa con una base relacional. El controlador de PostgreSQL es la pieza que sabe hablar con esa base, y ya viene instalado en el ambiente.

Servidor iceberg-postgres (azul). Es el servidor PostgreSQL del ambiente, que escucha en el puerto `5432`. Adentro está la base `origen` con la tabla `contribuyentes`.

Usuario lector (amarillo). Es la llave con que entras al origen. Solo puede hacer `SELECT` sobre esa tabla, así que el cuaderno no puede cambiar el origen aunque quiera.

La sentencia, parte por parte.

- `CREATE TEMPORARY VIEW origen_pg` crea una vista llamada `origen_pg`. Una vista no guarda datos, guarda la forma de ir a buscarlos. `TEMPORARY` quiere decir que vive solo en esta sesión del cuaderno. Si reinicias el kernel desaparece y hay que volver a crearla.
- `USING jdbc` dice que la vista lee por JDBC.
- `OPTIONS (...)` le pasa los datos de la conexión.
- `url 'jdbc:postgresql://iceberg-postgres:5432/origen'` es la dirección. `jdbc:postgresql` es el tipo de base, `iceberg-postgres` el servidor, `5432` el puerto y `origen` la base.
- `dbtable 'contribuyentes'` es la tabla del origen que se va a leer.
- `user 'lector'` es el usuario con que se entra.
- `password ''` es la clave, vacía. El origen confía en `lector` cuando la conexión viene de la red del ambiente, y solo para esta base. Así no queda ninguna clave escrita en el cuaderno.

```
sql
%%sql
-- Celda 1.2
CREATE TEMPORARY VIEW origen_pg USING jdbc OPTIONS (
  url      'jdbc:postgresql://iceberg-postgres:5432/origen',
  dbtable  'contribuyentes',
  user     'lector',
  password ''
)
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La vista quedó creada. Todavía no se leyó ni una fila del origen. Crear la vista solo deja anotado cómo llegar.

Celda 1.3 · Mirar lo que hay en el origen

La pregunta. ¿Qué hay en el origen antes de migrar?

Por qué ahora. Antes de copiar hay que saber qué se va a copiar, y ver que existe la columna de la fecha de actualización, que es la que permite la marca de agua.

En el almacén. Es mirar por la ventana la libreta vieja entera.

La sentencia, parte por parte.

- `SELECT *` pide todas las columnas.
- `FROM origen_pg` lee la vista, es decir, va al origen PostgreSQL en este momento.
- `ORDER BY rut` ordena por RUT, para que las filas salgan siempre en el mismo orden.

```
sql
%%sql
-- Celda 1.3
SELECT * FROM origen_pg ORDER BY rut
```

Lo que sale en pantalla.

```
text
rut      razon_social      segmento  actualizado_en
76000262-3  Boldo Importadora EIRL  MICRO    2026-09-01 08:00:00
76000503-7  Calafate Ferreteria EIRL  PEQUENA  2026-09-01 08:05:00
76002248-9  Andes Consultores SpA    MEDIANA  2026-09-01 08:10:00
76002652-2  Calafate Constructora S.A.  GRANDE  2026-09-01 08:15:00
76002830-4  Huemul Alimentos Ltda.   MICRO    2026-09-01 08:20:00
76004358-3  Ulmo Constructora Ltda.   PEQUENA  2026-09-01 08:25:00
76005022-9  Andes Distribuidora EIRL  MEDIANA  2026-09-01 08:30:00
76005135-7  Huemul Servicios SpA     GRANDE  2026-09-01 08:35:00
76005575-1  Copihue Consultores EIRL  MICRO    2026-09-01 08:40:00
76005701-0  Maiten Constructora Ltda.  PEQUENA  2026-09-01 08:45:00
76006049-6  Canelo Constructora S.A.  MEDIANA  2026-09-01 08:50:00
76007969-3  Maiten Alimentos EIRL    GRANDE  2026-09-01 08:55:00
76008296-1  Quillay Maquinarias Ltda.  MICRO    2026-09-01 09:00:00
76008516-2  Boldo Logistica SpA       PEQUENA  2026-09-01 09:05:00
76010063-3  Calafate Ferreteria EIRL  MEDIANA  2026-09-01 09:10:00
76010099-4  Rauli Transportes EIRL    GRANDE  2026-09-01 09:15:00
76011940-7  Canelo Servicios EIRL     MICRO    2026-09-01 09:20:00
76014257-3  Maiten Automotriz SpA     PEQUENA  2026-09-01 09:25:00
76015893-3  Lenga Consultores Ltda.   MEDIANA  2026-09-01 09:30:00
76016523-9  Andes Maquinarias EIRL    GRANDE  2026-09-01 09:35:00
20 filas.
```

Cómo se lee. Veinte contribuyentes, cada uno con su RUT, su razón social, su segmento y su fecha de actualización.

- `rut` es el identificador del contribuyente y va a ser la llave de la migración.
- `razon_social` es el nombre de la empresa.
- `segmento` es su tamaño, `MICRO`, `PEQUENA`, `MEDIANA` o `GRANDE`.
- `actualizado_en` es cuándo se creó o cambió la fila por última vez. Las veinte son del 1 de septiembre de 2026, cada una cinco minutos después de la anterior, de las 08:00:00 a las 09:35:00. Son horas del origen, sin zona horaria.

La fila más nueva es la de `76016523-9`, Andes Maquinarias EIRL, con `2026-09-01 09:35:00`. Esa va a ser la marca de agua.

Paso 2 · Carga inicial

Celda 2.1 · Partir de cero

La pregunta. ¿Hay una tabla de destino de una corrida anterior?

Por qué ahora. La carga inicial crea la tabla. Si ya existe de antes, la creación falla, así que primero se borra si está.

En el almacén. Es sacar del estante la libreta nueva a medio hacer de otro intento, si quedó alguna.

La sentencia, parte por parte.

- `DROP TABLE` borra la tabla.
- `IF EXISTS` hace que no se queje si la tabla no existe.
- `contribuyentes_lab16` es la tabla de destino, en `mi_espacio`.

```
sql
%%sql
-- Celda 2.1
DROP TABLE IF EXISTS contribuyentes_lab16
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Listo. Si la tabla estaba, ya no está. Si no estaba, no pasó nada.

Celda 2.2 · La foto completa

La pregunta. ¿Cómo se copia todo el origen a una tabla Iceberg?

Por qué ahora. Es la etapa de la carga inicial, la única que copia todo.

En el almacén. Es copiar la libreta vieja entera, de la primera a la última página, en la libreta nueva.

La sentencia, parte por parte.

- `CREATE TABLE contribuyentes_lab16` crea la tabla de destino.
- `USING iceberg` hace que sea una tabla Iceberg.
- `AS SELECT * FROM origen_pg` la llena con todo lo que devuelve la vista. Las columnas y sus tipos salen del resultado, así que no hace falta declararlos.

```
sql
%%sql
-- Celda 2.2
CREATE TABLE contribuyentes_lab16 USING iceberg AS
SELECT * FROM origen_pg
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla quedó creada y cargada en una sola sentencia. Spark leyó el origen por la vista y escribió los papelitos de la tabla nueva.

Celda 2.3 · Contar lo que llegó

La pregunta. ¿Llegaron todas las filas del origen?

Por qué ahora. Antes de seguir hay que confirmar que la foto está completa.

En el almacén. Es contar los renglones de la libreta nueva.

La sentencia, parte por parte.

- `count(*)` cuenta las filas.
- `AS contribuyentes` le pone nombre a la columna del resultado.
- `FROM contribuyentes_lab16` es la tabla de destino.

```
sql
%%sql
-- Celda 2.3
SELECT count(*) AS contribuyentes FROM contribuyentes_lab16
```

Lo que sale en pantalla.

```
text
contribuyentes
20
1 fila.
```

Cómo se lee. Veinte, los mismos del origen. La carga inicial está completa.

Celda 2.4 · Anotar la marca de agua

La pregunta. ¿Hasta qué fecha de actualización alcanzaste a traer?

Por qué ahora. Ese número es el que vas a usar en el paso 4 para saber qué falta. Si no se anota ahora, después no hay cómo saber qué se trajo.

En el almacén. Es el papel donde el que copia anota hasta qué hora alcanzó a copiar.

La sentencia, parte por parte.

- `max(actualizado_en)` busca la fecha de actualización más nueva.
- `AS marca_de_agua` le pone ese nombre a la columna.
- `FROM contribuyentes_lab16` la busca en el destino, que es lo que de verdad se trajo, y no en el origen, que pudo cambiar mientras tanto.

```
sql
%%sql
-- Celda 2.4
SELECT max(actualizado_en) AS marca_de_agua FROM contribuyentes_lab16
```

Lo que sale en pantalla.

```
text
marca_de_agua
2026-09-01 09:35:00
1 fila.
```

Cómo se lee. La marca de agua es `2026-09-01 09:35:00`. Todo lo que en el origen tenga una fecha posterior a esa es lo que falta. En este laboratorio la escribes a mano en las sentencias de los pasos 4 y 5. En producción se guarda en una tabla de control y el proceso la lee solo.

Paso 3 · El origen sigue vivo

Este paso no tiene celdas en el cuaderno. Lo que haces es en una terminal.

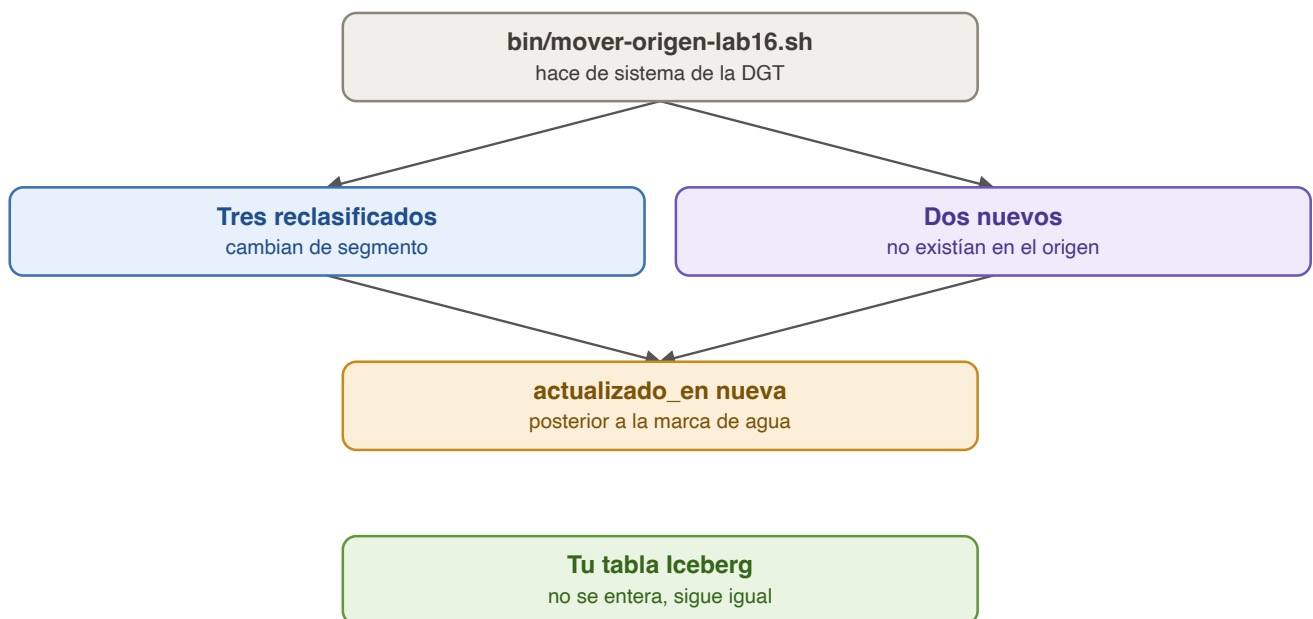
La pregunta. ¿Qué pasa en el origen mientras tú terminas la carga inicial?

Por qué ahora. Sin este paso la migración parecería terminada. Con él se ve por qué la carga inicial no alcanza.

En el almacén. Mientras copiabas, en la libreta vieja se anotaron dos clientes nuevos y se corrigió el tamaño de otros tres.

Para simularlo, abre una terminal en la carpeta del repositorio y corre el proceso que mueve el origen.

```
bash
bin/mover-origen-lab16.sh
```



bin/mover-origen-lab16.sh (gris). Es un script del repositorio que escribe directo en el origen PostgreSQL, como lo haría el sistema que lo alimenta. El cuaderno no puede hacerlo, porque su usuario solo lee.

Tres reclasificados (azul). Tres contribuyentes que ya tenías cambian de segmento.

Dos nuevos (morado). Entran dos contribuyentes que no estaban.

actualizado_en nueva (amarillo). Los cinco quedan con una fecha de actualización posterior a tu marca de agua. Es lo único que vas a tener para encontrarlos.

Tu tabla Iceberg (verde). No cambia. Sigue con sus veinte filas, y así tiene que ser. Una migración no se entera de los cambios, los va a buscar.

En el cuaderno no vas a ver nada. Si repites el laboratorio, antes deja el origen como estaba con `bin/mover-origen-lab16.sh --reset`.

Paso 4 · Detectar qué cambió

Celda 4.1 · Lo que se movió desde la marca de agua

La pregunta. ¿Qué filas del origen cambiaron o aparecieron después de la carga inicial?

Por qué ahora. Es la detección de cambios, y con la marca de agua es una sola condición.

En el almacén. Es buscar en la libreta vieja solo lo que se anotó después de la hora que dice tu papel.

La sentencia, parte por parte.

- `SELECT *` trae todas las columnas.
- `FROM origen_pg` lee el origen en este momento, con los cambios del paso 3.
- `WHERE actualizado_en > TIMESTAMP '2026-09-01 09:35:00'` se queda con las filas cuya fecha de actualización es estrictamente posterior a la marca de agua. `TIMESTAMP '...'` convierte el texto en una fecha con hora. El signo mayor, y no mayor o igual, es porque la fila de la marca misma ya la trajiste.
- `ORDER BY rut` ordena por RUT.

```
sql
%%sql
-- Celda 4.1
SELECT * FROM origen_pg
WHERE actualizado_en > TIMESTAMP '2026-09-01 09:35:00'
ORDER BY rut
```

Lo que sale en pantalla.

```
text
rut      razon_social      segmento  actualizado_en
76000262-3  Boldo Importadora EIRL  PEQUENA   2026-09-15 12:00:00
76002248-9  Andes Consultores SpA  GRANDE    2026-09-15 12:00:00
76005135-7  Huemul Servicios SpA  MEDIANA   2026-09-15 12:00:00
77129445-1  Lenga Transportes SpA  MICRO     2026-09-15 12:00:00
78440174-7  Copihue Alimentos EIRL  PEQUENA   2026-09-15 12:00:00
5 filas.
```

Cómo se lee. Cinco filas, todas con `2026-09-15 12:00:00`, la hora en que el script movió el origen.

- `76000262-3`, `76002248-9` y `76005135-7` ya estaban en la carga inicial. Cambiaron de segmento. Boldo Importadora pasó de `MICRO` a `PEQUENA`, Andes Consultores de `MEDIANA` a `GRANDE` y Huemul Servicios de `GRANDE` a `MEDIANA`.
- `77129445-1`, Lenga Transportes, y `78440174-7`, Copihue Alimentos, son nuevos. No estaban en los veinte del paso 1.

La fecha es siempre la misma porque el script la deja fija, para que la salida sea igual cada vez que se repite el laboratorio.

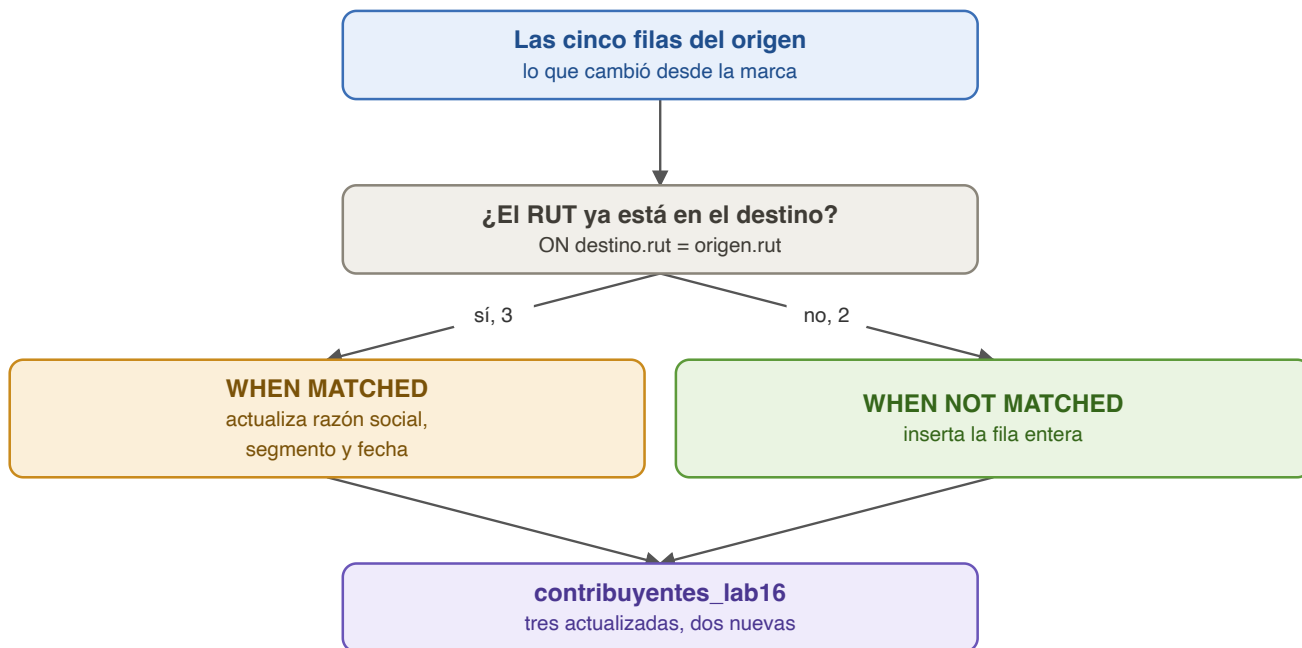
Paso 5 · Carga incremental

Celda 5.1 · El MERGE

La pregunta. ¿Cómo se meten esas cinco filas sin duplicar las tres que ya estaban?

Por qué ahora. Un `INSERT` dejaría a los tres reclasificados dos veces, con el segmento viejo y con el nuevo. Hace falta una sentencia que actualice lo que ya está e inserte lo que no.

En el almacén. Es pasar cada renglón nuevo a la libreta nueva mirando primero si ese cliente ya tiene renglón. Si lo tiene, se corrige. Si no, se agrega uno.



Las cinco filas del origen (azul). Son las que devolvió la detección de cambios.

¿El RUT ya está en el destino? (gris). El **MERGE** compara cada fila que llega con las del destino por la llave, el RUT.

WHEN MATCHED (amarillo). Si el RUT ya está, actualiza la razón social, el segmento y la fecha de actualización.

WHEN NOT MATCHED (verde). Si el RUT no está, inserta la fila completa.

contribuyentes_lab16 (morado). Al final la tabla tiene los tres reclasificados corregidos y los dos nuevos agregados.

La sentencia, parte por parte.

- **MERGE INTO contribuyentes_lab16 AS destino** es la tabla que se modifica, con el apodo **destino**.
- **USING (SELECT * FROM origen_pg WHERE actualizado_en > TIMESTAMP '2026-09-01 09:35:00') AS origen** son las filas que llegan, las mismas cinco de la celda 4.1, con el apodo **origen**.
- **ON destino.rut = origen.rut** dice cuándo dos filas son el mismo contribuyente. Es la llave.
- **WHEN MATCHED THEN UPDATE SET** dice qué hacer si el contribuyente ya estaba.
- **destino.razon_social = origen.razon_social, destino.segmento = origen.segmento y destino.actualizado_en = origen.actualizado_en** copian del origen al destino las tres columnas que pueden cambiar. El RUT no se toca, porque es la llave.
- **WHEN NOT MATCHED THEN INSERT *** dice qué hacer si no estaba, insertar la fila con todas sus columnas.

```
sql
%%sql
-- Celda 5.1
MERGE INTO contribuyentes_lab16 AS destino
USING (
    SELECT * FROM origen_pg
    WHERE actualizado_en > TIMESTAMP '2026-09-01 09:35:00'
) AS origen
ON destino.rut = origen.rut
WHEN MATCHED THEN UPDATE SET
    destino.razon_social = origen.razon_social,
    destino.segmento = origen.segmento,
    destino.actualizado_en = origen.actualizado_en
WHEN NOT MATCHED THEN INSERT *
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. El `MERGE` se ejecutó. No dice cuántas filas actualizó ni cuántas insertó. Eso se comprueba en las celdas que siguen.

Celda 5.2 · Contar otra vez

La pregunta. ¿Cuántas filas tiene ahora el destino?

Por qué ahora. Si el `MERGE` hizo bien su trabajo, los tres reclasificados no suman filas y los dos nuevos sí.

En el almacén. Es volver a contar los renglones de la libreta nueva.

La sentencia, parte por parte.

- `count(*)` cuenta las filas.
- `AS contribuyentes` le pone nombre a la columna.
- `FROM contribuyentes_lab16` es la tabla de destino.

```
sql
%%sql
-- Celda 5.2
SELECT count(*) AS contribuyentes FROM contribuyentes_lab16
```

Lo que sale en pantalla.

```
text
contribuyentes
22
1 fila.
```

Cómo se lee. Veintidós. Son los veinte de la carga inicial más los dos nuevos. Los tres reclasificados se actualizaron y no agregaron filas.

Paso 6 · Validar

Celda 6.1 · Conteo y suma de control

La pregunta. ¿Dicen lo mismo el origen y el destino?

Por qué ahora. Contar no basta. Dos tablas pueden tener el mismo número de filas y decir cosas distintas, por ejemplo si un segmento quedó mal copiado. Hace falta un número que cambie si cambia cualquier dato.

En el almacén. Es contar los renglones de las dos libretas y, además, sumar un número que se saca de cada renglón. Si un solo renglón está distinto, las sumas no coinciden.

La sentencia, parte por parte.

- `SELECT (...) AS filas_origen, (...) AS filas_iceberg, (...) AS control_origen, (...) AS control_iceberg` arma una sola fila con cuatro números, cada uno calculado por una subconsulta entre paréntesis.
- `SELECT count(*) FROM origen_pg` cuenta las filas del origen.
- `SELECT count(*) FROM contribuyentes_lab16` cuenta las filas del destino.
- `concat(rut, razon_social, segmento)` pega en un solo texto el RUT, la razón social y el segmento de cada fila.
- `crc32(...)` calcula sobre ese texto un número, siempre el mismo para el mismo texto y casi siempre distinto si cambia una sola letra. Se llama suma de control.
- `sum(...)` suma esos números de todas las filas. El orden de las filas no importa para una suma, así que los dos lados se pueden comparar aunque estén ordenados distinto.
- `FROM origen_pg` y `FROM contribuyentes_lab16` hacen el mismo cálculo en el origen y en el destino.

```
sql
%%sql
-- Celda 6.1
SELECT
    (SELECT count(*) FROM origen_pg) AS filas_origen,
    (SELECT count(*) FROM contribuyentes_lab16) AS
filas_iceberg,
    (SELECT sum(crc32(concat(rut, razon_social, segmento))) FROM origen_pg) AS control_origen,
    (SELECT sum(crc32(concat(rut, razon_social, segmento))) FROM contribuyentes_lab16) AS
control_iceberg
```

Lo que sale en pantalla.

```
text
filas_origen  filas_iceberg  control_origen  control_iceberg
22            22            39995702816     39995702816
1 fila.
```

Cómo se lee. Los cuatro números cuadran de a pares. Veintidós filas en el origen y veintidós en el destino, y la misma suma de control, 39995702816, en los dos lados. La migración está validada con números, no con la palabra de nadie.

La suma de control no incluye `actualizado_en`. Mide lo que importa del contribuyente, que es quién es y en qué segmento está.

Paso 7 · Lo que quedó escrito

Celda 7.1 · Las páginas de la migración

La pregunta. ¿Qué dejó anotado la migración en la libreta?

Por qué ahora. Si mañana alguien pregunta qué trajo cada carga, la respuesta está en las páginas de la tabla.

En el almacén. Es mirar la lista de páginas de la libreta nueva.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide el número de cada página, la hora en que se colgó, en UTC, y qué operación fue.
- `FROM mi_espacio.contribuyentes_lab16.snapshots` es la vista de sistema con la lista de páginas. El nombre tiene tres partes, el espacio, la tabla y la vista.
- `ORDER BY committed_at` ordena de la página más vieja a la más nueva.

```
sql
%%sql
-- Celda 7.1
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.contribuyentes_lab16.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
2550665139341160300  2026-09-25 18:11:43.816  append
7138320268844862618  2026-09-25 18:11:52.668  overwrite
2 filas.
```

Cómo se lee. Dos páginas, una por cada carga.

- `2550665139341160300`, a las 18:11:43.816 UTC, las 15:11 en Chile, es la carga inicial. `append` quiere decir que se agregaron filas.
- `7138320268844862618`, a las 18:11:52.668 UTC, es la carga incremental. Sale como `overwrite` porque el `MERGE` reescribió el papelito donde estaban los tres reclasificados, además de agregar los nuevos.

Con esos dos números se puede pedir la tabla como quedó después de cada carga, con `VERSION AS OF`.

Preguntas frecuentes

¿Qué pasa si en el origen se borra una fila?

La marca de agua no la detecta. Una fila borrada no queda con fecha nueva, simplemente deja de estar. Para eso el origen tiene que marcar los borrados en vez de borrarlos, con una columna que diga que la fila está dada de baja y su fecha, o hay que comparar las llaves de los dos lados de vez en cuando. La suma de control del paso 6 sí lo notaría, porque el origen tendría una fila menos.

¿Y si dos filas tienen exactamente la misma fecha que la marca de agua?

Por eso la condición es mayor y no mayor o igual, y por eso la marca se saca del destino. Todo lo que tenía esa fecha ya se trajo. El riesgo es otro. Si el origen anota una fila con una fecha anterior a la marca después de la carga, por un reloj atrasado o una transacción lenta, esa fila no se detecta. En producción se suele restar un margen a la marca, unos minutos, y dejar que el `MERGE` absorba lo repetido.

¿Por qué `MERGE` y no `INSERT`?

Porque `INSERT` agrega siempre. Con `INSERT` los tres reclasificados quedarían dos veces, con el segmento viejo y con el nuevo, y la tabla tendría veinticinco filas en vez de veintidós. `MERGE` decide fila por fila por la llave.

¿Por qué el usuario `lector` no tiene clave?

Porque la celda que abre el origen queda escrita en el cuaderno y en el repositorio, y una clave ahí sería una credencial a la vista de todos. En este ambiente PostgreSQL confía en `lector` solo cuando la conexión viene de la red del ambiente y solo para la base `origen`, y `lector` solo puede leer una tabla de datos inventados. En una plataforma real la clave vendría de un almacén de secretos o del sistema de autenticación de la institución, nunca escrita en la celda.

¿Dónde se guarda la marca de agua en producción?

En una tabla de control, con una fila por tabla migrada y la fecha de su última carga. El proceso la lee al empezar, carga lo que falta y, si todo salió bien, la actualiza. Si el proceso falla a mitad, la marca no se mueve y la próxima corrida vuelve a traer lo mismo, que el `MERGE` absorbe sin duplicar.

¿La suma de control puede coincidir aunque los datos sean distintos?

Es posible, pero muy improbable. `crc32` da un número de 32 bits, y que dos cambios distintos se compensen justo en la suma es raro. Para una validación más fuerte se usan funciones como `md5` o `sha2`, o se compara columna por columna.

¿Por qué la vista es temporal?

Porque no hace falta guardarla en el catálogo. Es una ventana para esta sesión. Si reinicias el kernel, vuelve a correr la celda 1.2 antes de las que usan `origen_pg`.

¿Qué pasa si corro el laboratorio dos veces sin dejar el origen como estaba?

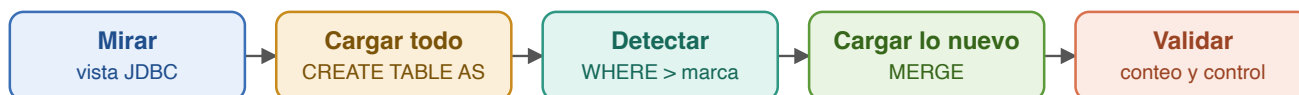
La carga inicial traería veintidós contribuyentes, la marca de agua sería `2026-09-15 12:00:00` y la detección del paso 4 no devolvería nada. Para que salga como en esta guía, antes corre `bin/mover-origen-lab16.sh -reset`.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Una migración con el origen vivo son cinco etapas, y hoy hiciste las cinco con una tabla y dos cargas. Entre una tabla y cientos no cambia el procedimiento. Cambia que la marca de agua de cada tabla se guarda en una tabla de control y que las cargas se encadenan y se vigilan.

En el almacén. Copiaste la libreta vieja una vez, anotaste hasta qué hora llegaste, y la vez siguiente pasaste solo lo nuevo, corrigiendo lo que ya tenías y agregando lo que no. Al final contaste y sumaste las dos libretas, y dieron lo mismo.



Mirar (azul). La vista JDBC abre el origen sin copiarlo.

Cargar todo (amarillo). La carga inicial copia todo una vez y deja la marca de agua.

Detectar (verde agua). Una condición sobre `actualizado_en` encuentra lo que cambió.

Cargar lo nuevo (verde). El `MERGE` actualiza lo que estaba e inserta lo que no.

Validar (coral). El conteo y la suma de control prueban que los dos lados dicen lo mismo.

Los números de la solución ejecutada.

Momento	Origen	Destino	Qué pasó
Carga inicial	20	20	marca de agua 2026-09-01 09:35:00
Después de mover el origen	22	20	tres reclasificados y dos nuevos
Después del <code>MERGE</code>	22	22	suma de control 39995702816 en los dos lados

Lo que no cambia nunca es que el origen sigue vivo mientras migras, y que por eso la carga inicial no alcanza.